

Rosenblatt Derivation

```
Warning: package 'ggplot2' was built under R version 4.5.3
```

```
Warning: package 'tidyr' was built under R version 4.5.3
```

```
Warning: package 'readr' was built under R version 4.5.3
```

```
Warning: package 'purrr' was built under R version 4.5.3
```

```
Warning: package 'stringr' was built under R version 4.5.3
```

```
Warning: package 'forcats' was built under R version 4.5.3
```

```
Warning: package 'lubridate' was built under R version 4.5.3
```

```
Warning: package 'invgamma' was built under R version 4.5.2
```

```
Warning: package 'patchwork' was built under R version 4.5.3
```

```
Warning: package 'SHT' was built under R version 4.5.3
```

Rosenblatt transformation

The Rosenblatt transformation is the multivariate extension of the probability integral transformation. It applies the conditional CDF of each element given the elements before it. This removes the modeled dependence sequentially. If the proposed joint distribution is correct, the resulting transformed data are independent Uniform(0, 1) random variables.

Let

$$\mathbf{X} = (X_1, \dots, X_p)^T$$

be a continuous random vector with joint distribution function $F(x_1, \dots, x_k)$.

We define the Rosenblatt transformation as follows:

$$\begin{aligned}
U_1 &= F_{X_1}(X_1), \\
U_2 &= F_{X_2|X_1}(X_2 | X_1), \\
U_3 &= F_{X_3|X_1, X_2}(X_3 | X_1, X_2), \\
&\vdots \\
U_p &= F_{X_p|X_1, \dots, X_{p-1}}(X_p | X_1, \dots, X_{p-1}).
\end{aligned}$$

If the assumed joint distribution is correct and its parameters are known, then

$$U_1, \dots, U_p \stackrel{\text{ind}}{\sim} \text{Uniform}(0, 1).$$

Each random variable is transformed using its distribution conditional on all the random variables that came before it.

Skewed tensor variate distributions

The tensor skew- t , variance-gamma, inverse-Gaussian, and generalized hyperbolic distributions implemented in `tensortools` share the representation

$$\mathcal{X} = \mathcal{M} + W\mathcal{A} + \sqrt{W}Z,$$

where $\mathcal{A}$ is the skew tensor, W is a positive scalar mixing variable, and

$$Z \sim \text{TVN}(\mathbf{0}, \Sigma_1, \dots, \Sigma_D)$$

is independent of W . Conditional on $W = w$,

$$X \sim N(\mathcal{M} + w\mathcal{A}, w[\Sigma_D \otimes \dots \Sigma_1])$$

Univariate Skewed-t

Take $X = \mu + W\mathcal{A} + \sigma\sqrt{W}Z$, $Z \sim N(0, 1)$ where $W > 0$ is a latent mixing variable. Conditional on $W = w$, $X|W = w \sim N(\mu + wa, w\sigma^2)$. That is, if we knew the value w of the latent mixing variable, X would follow a normal distribution with mean $\mu + wa$ and variance $w\sigma^2$.

In the univariate case, the Rosenblatt transformation is the same as the probability integral transform.

First, we can obtain the marginal CDF of X by integrating out the dependence on W .

$$\begin{aligned}
F_X(x) &= P(X \leq x) \\
&= \int_0^\infty P(X \leq x | W = w) f_W(w) dw \\
&= \int_0^\infty f_{X|W}(x|w) f_W(w) dw \\
&= \int_0^\infty \Phi\left(\frac{x - \mu - wa}{\sigma\sqrt{w}}\right) f_W(w) dw
\end{aligned}$$

Then, we can apply this CDF to the draws. If the model is correct, $U \sim \text{Uniform}(0, 1)$.

```
nu <- 4 # generate univariate values
mu <- 1
variance <- 2
skew <- 3

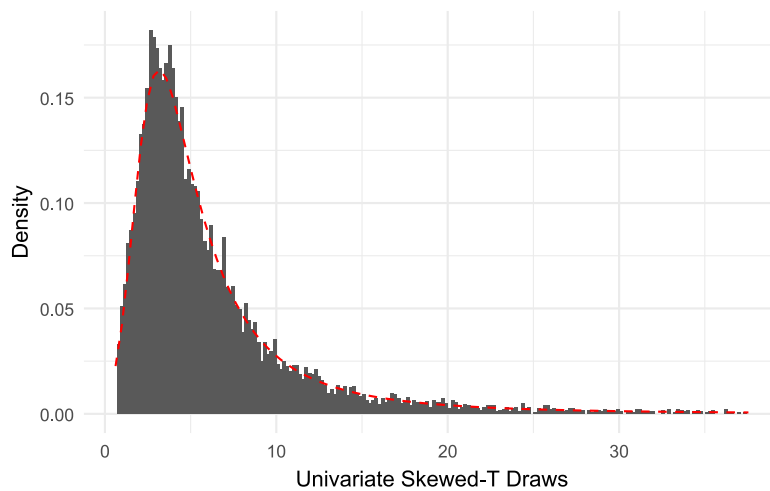
# create univariate draws
univar_skewt <- rtskewt(n = 1e4, mu = mu, sigmas = list(matrix(variance)),
                      skew = skew, nu = nu)

# define vectorized skewt for ggplot
dtskewt_vec <- function(x, mu, skew, sigmas, nu, log = FALSE) {
  if (length(mu) != 1L || length(skew) != 1L ||
      length(sigmas) != 1L ||
      !all(dim(sigmas[[1]]) == c(1L, 1L))) {
    stop("dtskewt_vec() is only for the univariate case.")
  }

  vapply(
    x,
    function(xi) {
      dtskewt(x = xi, mu = mu, skew = skew, sigmas = sigmas,
              nu = nu, log = log)
    },
    numeric(1)
  )
}
```

Below, you can see the plot of the skewed-t distribution. The randomly generated draws are plotted with an overlay of the expected distribution.

```
ggplot() +
  geom_histogram(aes(x = univar_skewt, y = after_stat(density)),
                bins = 200) +
  geom_function(fun = dtskewt_vec,
               args = list(mu = mu, skew = skew,
                           sigmas = list(matrix(variance)), nu = nu),
               color = "red", lty = 2) +
  xlim(quantile(univar_skewt, 0.01), quantile(univar_skewt, 0.99)) +
  labs(x = "Univariate Skewed-T Draws", y = "Density")
```



We can apply the Rosenblatt transformation to transform the draws from the skewed-t distribution to a uniform. Since these draws are univariate, the Rosenblatt is identical to the Probability Integral Transformation in this case. We apply the CDF to the draws to obtain a uniform.

```
skew_t_cdf <- function(x, mu, variance, skew, nu) {
  shape <- nu/2
  rate <- nu/2

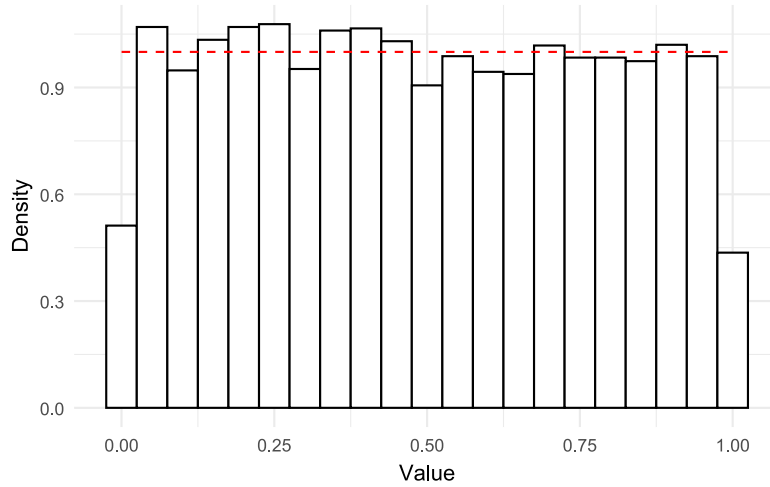
  integrand <- function(w) {
    pnorm((x - mu - w * skew) / sqrt(w * variance)) * # f(x|w)
    dinvgamma(w, shape = shape, rate = rate) # f(w)
  }

  # integrate out dependence on w
  integrate(integrand, lower = 0, upper = Inf)$value
}

res <- numeric(length(univar_skewt))

for (i in seq_along(univar_skewt)) { # apply to each draw
  res[i] <- skew_t_cdf(x = univar_skewt[i], mu = mu, variance = variance,
                      skew = skew, nu = nu)
}

ggplot() +
  geom_histogram(aes(x = res, y = after_stat(density)), binwidth = 0.05,
                color = "black", fill = "white") +
  geom_function(fun = dunif, color = "red", lty = 2) +
  labs(x = "Value", y = "Density")
```



Multivariate Skewed-t

Now suppose we observe a bivariate skewed-t.

$$X = \mu + W\mathbf{a} + \sqrt{W}\mathbf{Z}, \quad Z \sim N_2(0, \Sigma)$$

where

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$$

Now we need the Rosenblatt to compute $U_1 = F_{X_1}(x_1)$ and $U_2 = F_{X_2|X_1}(x_2|x_1)$.

Multivariate Skewed-t X_1

First, consider X_1 . Conditional on $W = w$, $X_1|W = w \sim N(\mu_1 + wa_1, w\sigma_1^2)$. This is the same as the univariate case we defined above.

```
nu <- 4 # generate bivariate values
mu <- c(1, 2)
variance <- matrix(data = c(1, 0.5, 0.5, 3), nrow = 2)
skew <- c(0.2, 0.4)

bivar_skewt <- rtskewt(n = 1e4, mu = mu, sigmas = list(variance),
                      skew = skew, nu = nu)
```

Multivariate Skewed-t X_2

Once we have observed $X_1 = x_1$, we have more information about the mixing variable W . Instead of using $f_W(w)$, we want to find $f_{W|X_1}(w|x_1)$.

We can use Bayes' rule to compute

$$f_{W|X_1}(w|x_1) \propto f_{X_1|W}(x_1|w)f_W(w)$$

We can then compute U_2 using the updated CDF of X_2 .

$$F_{X_2|X_1}(x_2|x_1) = \int_0^\infty F_{X_2|X_1,W}(x_2|x_1, w) f_{W|X_1}(w|x_1) dw$$

We can apply the CDF to the draws. If the model is correct, they should become Uniform(0, 1).

```
n <- n_draws(bivar_skewt)

L <- t(chol(variance))

b <- drop(forwardsolve(L, skew))

centered <- bivar_skewt - mu

Y <- t(forwardsolve(L, t(centered)))

conditional_u2 <- function(x1, x2, b1, b2, nu) {
  weight <- function(w) {
    dnorm(x1, mean = w * b1, sd = sqrt(w)) *
    dinvgamma(w, shape = nu / 2, rate = nu / 2)
  }

  numerator <- integrate(
    function(w) {
      pnorm(x2, mean = w * b2, sd = sqrt(w)) * weight(w)
    },
    lower = 0,
    upper = Inf
  )$value

  denominator <- integrate(
    weight,
    lower = 0,
    upper = Inf
  )$value

  numerator / denominator
}

u1 <- vapply(
  seq_len(n),
  function(i) {
    skew_t_cdf(
```

```

      x      = bivar_skewt[i][1],
      mu      = mu[1],
      variance = variance[1, 1],
      skew     = skew[1],
      nu      = nu
    )
  },
  numeric(1)
)

# integrate() itself is not vectorized over observations
u2 <- vapply(
  seq_len(n),
  function(i) {
    conditional_u2(
      x1 = Y[i, 1],
      x2 = Y[i, 2],
      b1 = b[1],
      b2 = b[2],
      nu = nu
    )
  },
  numeric(1)
)

res_whitened <- cbind(u1 = u1, u2 = u2)

```

In the plots below, we can see how the Rosenblatt transformation works. The first plot contains the original draws. The second plot is after the Rosenblatt has transformed X_1 into U_1 , which should now be uniformly distributed along that 1st dimension. The third plot is after the Rosenblatt has transformed X_2 into U_2 . Now all of the points are uniformly distributed in the space $\mathbb{R}^2$.

```

draws_orig <- tibble(x1 = rep(0, 1e4), x2 = rep(0, 1e4))

for(i in 1:1e4) {
  draws_orig$x1[i] <- bivar_skewt[i][1]
  draws_orig$x2[i] <- bivar_skewt[i][2]
}

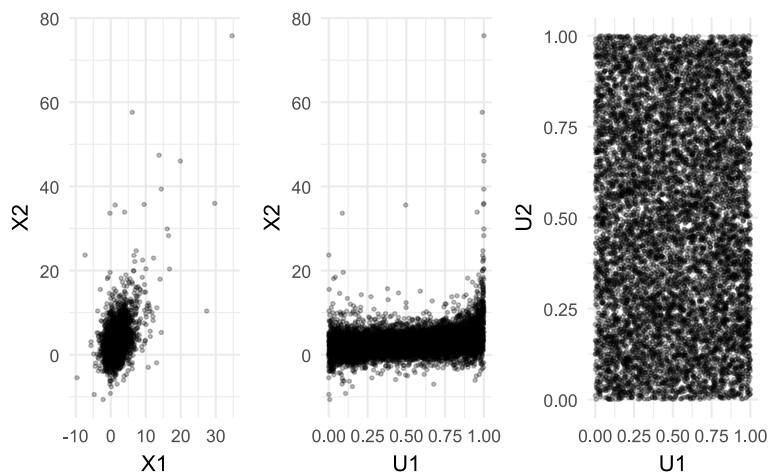
(draws_orig |>
  ggplot() +
  geom_point(aes(x = x1, y = x2), size = 0.6, alpha = 0.25) +
  theme_minimal() +

```

```

  labs(x = "X1", y = "X2")) +
(draws_orig |>
  ggplot() +
  geom_point(aes(x = u1, y = x2), size = 0.6, alpha = 0.25) +
  theme_minimal() +
  labs(x = "U1", y = "X2")) +
(draws_orig |>
  ggplot() +
  geom_point(aes(x = u1, y = u2), size = 0.6, alpha = 0.25) +
  theme_minimal() +
  labs(x = "U1", y = "U2"))

```



Wrong Parameters

We can intentionally run the Rosenblatt transformation with wrong parameters to see its effect. The true variance used to generate the draws was

$$\Sigma = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 3 \end{pmatrix}$$

Now, we replace $\Sigma_{1,1}$ with 3.

```

#true variance: c(1, 0.5, 0.5, 3)
#replace true parameter value
wrong_variance <- matrix(data = c(3, 0.5, 0.5, 3), nrow = 2)

L <- t(chol(wrong_variance))

b <- drop(forwardsolve(L, skew))

```

```

centered <- bivar_skewt - mu

Y <- t(forwardsolve(L, t(centered)))

u1_wrong <- vapply(
  seq_len(n),
  function(i) {
    skew_t_cdf(
      x      = bivar_skewt[i][1],
      mu     = mu[1],
      variance = wrong_variance[1, 1],
      skew   = skew[1],
      nu     = nu
    )
  },
  numeric(1)
)

# integrate() itself is not vectorized over observations
u2_wrong <- vapply(
  seq_len(n),
  function(i) {
    conditional_u2(
      x1 = Y[i, 1],
      x2 = Y[i, 2],
      b1 = b[1],
      b2 = b[2],
      nu = nu
    )
  },
  numeric(1)
)

res_whitened_wrong <- cbind(u1 = u1_wrong, u2 = u2_wrong)

```

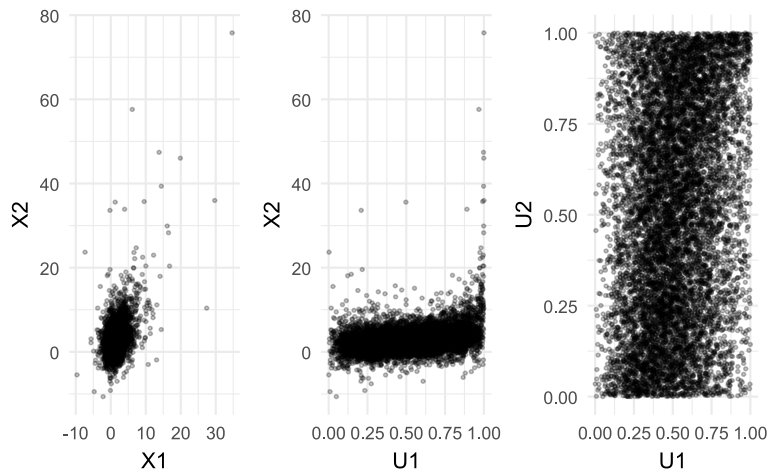
Below is the effect of applying the Rosenblatt distribution with the incorrect variance term. The final draws plotted in the U_1 by U_2 are not uniform across the space $\mathbb{R}^2$.

```

(draws_orig |>
  ggplot() +
  geom_point(aes(x = x1, y = x2), size = 0.6, alpha = 0.25) +
  theme_minimal() +
  labs(x = "X1", y = "X2")) +

```

```
(draws_orig |>
  ggplot() +
  geom_point(aes(x = u1_wrong, y = x2), size = 0.6, alpha = 0.25) +
  theme_minimal() +
  labs(x = "U1", y = "X2")) +
(draws_orig |>
  ggplot() +
  geom_point(aes(x = u1_wrong, y = u2_wrong), size = 0.6, alpha = 0.25) +
  theme_minimal() +
  labs(x = "U1", y = "U2"))
```



Formal Diagnostic

For now, assume we have the correct parameter values. The next issue is we need to assess whether the transformed draws are uniform on the unit hypercube. This can be done using an Interpoint Distances test (Yang and Modarres, 2014).

We define the interpoint distances as

$$\sum_{i < j}^n ||X_i - X_j||^2$$

.

We can define the sample mean of the interpoint distances as

$$\bar{\delta} = \frac{1}{\binom{n}{2}} \sum_{i < j}^n ||X_i - X_j||^2$$

And the sample variance of the interpoint distances.

$$\bar{S} = \frac{1}{\binom{n}{2}} \sum_{i < j}^n \left(|X_i - X_j|^2 - \frac{d}{6} \right)^2$$

We want to apply the Central Limit Theorem, so we need to find the variances.

$$\text{Var}(\hat{\delta}) = \frac{d(2n+3)}{90n(n-1)}$$

and

$$\text{Var}(\bar{S}) = \begin{cases} \frac{1}{\binom{n}{2}} \left(\frac{1}{56700} [989 + 202(n-2)] \right), & d = 2 \\ \frac{1}{\binom{n}{2}} \left(\frac{1}{1050} [37 + 6(n-2)] \right), & d = 3 \\ \frac{1}{\binom{n}{2}} \left(\frac{49d^2}{16200} + \frac{101d}{37800} + 2(n-2) \left[\frac{d^2}{16200} + \frac{29d}{37800} \right] \right), & d \geq 4 \end{cases}$$

Let $\sigma_{\bar{\delta}} = \sqrt{\text{Var}(\bar{\delta})}$ and $\sigma_{\bar{S}} = \sqrt{\text{Var}(\bar{S})}$.

Then with the Central Limit Theorem for U_processes, we can say that

$$\frac{\bar{\delta} - d/6}{\sigma_{\bar{\delta}}} \xrightarrow{D} N(0, 1)$$

and

$$\frac{\bar{S} - 7d/180}{\sigma_{\bar{S}}} \xrightarrow{D} N(0, 1)$$

We define

$$Q_1 = \frac{(\bar{\delta} - d/6)^2}{\text{Var}(\bar{\delta})}$$

and

$$Q_2 = \frac{(\bar{S} - 7d/180)^2}{\text{Var}(\bar{S})}$$

.

Then $Q_1 \xrightarrow{D} \chi_1^2$ and $Q_2 \xrightarrow{D} \chi_1^2$.

Since the mean $\bar{\delta}$ and the variance $\bar{S}$ of asymptotically normal variables are independent, we can also define $Q_3 = Q_1 + Q_2 \xrightarrow{D} \chi_2^2$.

Test Method

This method has been implemented in the package **SHT** (Statistical Hypothesis Testing Toolbox) as the function `unif.2017YMi`. However, it is inefficient as it constructs the entire $n \times n$ matrix of

pairwise distances. The rewritten function below instead takes advantage of vectorized operations to be significantly faster.

```
interpoint_dist <- function(X) {
  X_array <- if (inherits(X, "tensor")) unclass(X) else X
  X_dims <- dim(X_array)

  if (is.null(X_dims) || length(X_dims) < 2L) {
    stop("X must be a matrix or an array with observations on its first axis.")
  }

  n <- X_dims[1L]
  d <- prod(X_dims[-1L])

  if (n < 2 || d < 2)
    stop("Need at least two observations and two dimensions.")

  X_mat <- matrix(as.numeric(X_array), nrow = n, ncol = d) # matricize data

  pairs <- choose(n, 2) #every possible combination
  fac <- 1 / pairs

  values_sq <- rowSums(X_mat^2) #computes x_{i}^2, each obs squared
  obs <- colSums(X_mat) #computes x_{ik}, total of coord across all samples

  # Sum of squared pairwise distances
  sum_pairwise <- n * sum(values_sq) - sum(obs^2)

  # Sum of fourth powers of pairwise distances
  values_four <- sum(values_sq^2)
  v <- drop(crossprod(values_sq, X_mat))

  # Only the squared Frobenius norm is needed. Use the smaller Gram matrix:
  # ||X'X||_F^2 = ||XX'||_F^2.
  cross_sq <- if (d <= n) {
    cross <- crossprod(X_mat)
    sum(cross * cross)
  } else {
    gram_obs <- tcrossprod(X_mat)
    sum(gram_obs * gram_obs)
  }

  S2 <- (n - 2) * values_four + sum(values_sq)^2 -
    4 * (sum(obs * v) - values_four) +
```

```

      2 * (cross_sq - values_four)

q1_num <- fac * sum_pairwise

c <- d / 6

q2_num <- fac * (S2 - 2 * c * sum_pairwise + pairs * c^2)

q1_denom <- d * (2 * n + 3) / (90 * n * (n - 1))

q2_denom <- if (d == 2) {
  ((989 + 202 * (n - 2)) / 56700) * fac
} else if (d == 3) {
  ((37 + 6 * (n - 2)) / 1050) * fac
} else {
  (49 * d^2 / 16200 +
   101 * d / 37800 +
   2 * (n - 2) * (d^2 / 16200 + 29 * d / 37800)) * fac
}

Q1 <- (q1_num - d / 6)^2 / q1_denom
Q2 <- (q2_num - 7 * d / 180)^2 / q2_denom
Q3 <- Q1 + Q2

tibble(Q1 = Q1, Q2 = Q2, Q3 = Q3,
       p_Q1 = pchisq(Q1, df = 1, lower.tail = FALSE),
       p_Q2 = pchisq(Q2, df = 1, lower.tail = FALSE),
       p_Q3 = pchisq(Q3, df = 2, lower.tail = FALSE))
}

bench::mark(unname(SHT::unif.2017YMi(res_whitened, "Q1")$statistic),
            interpoint_dist(res_whitened)$Q1)

```

Warning: Some expressions had a GC in every iteration; so filtering is disabled.

```

# A tibble: 2 × 6
  expression                min median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>              <bch:t> <bch:>    <dbl>   <bch:byt>    <dbl>
1 "unname(SHT::unif.2017YMi(res_whi...  4.72s  4.72s    0.212   5.59GB     1.48
2 "interpoint_dist(res_whitened)$Q1" 904.8µs 1.29ms   624.     1.31MB     3.99

```

First, we can test that this metric works when we generate truly uniform values. As expected, the rejection rate is about 0.05. However, Q_3 had a slightly higher rejection rate. Q_1 and Q_2 are both asymptotically distributed as a χ^2_1 . Q_3 is defined as the sum of Q_1 and Q_2 , so it depends on the joint behavior. This may lead to the finite-sample distribution of Q_3 to deviate from its asymptotic distribution of χ^2_2 . Due to this behavior, I will use the p -values from Q_1 and Q_2 as the main diagnostic values for rejection, but will still report Q_3 in case it is of interest.

```
sim_uniform <- function(i) {
  p <- length(mu)

  X <- matrix(
    runif(1e4 * p),
    nrow = 1e4,
    ncol = p
  )

  interpoint_dist(X)
}

res_uniform <- map_dfr(1:1e4, sim_uniform)

saveRDS(res_uniform, "res_uniform_interpoint.rds")
```

```
res_uniform <- readRDS("res_uniform_interpoint.rds")

c(Q1_reject = mean(res_uniform$p_Q1 < 0.05),
  Q2_reject = mean(res_uniform$p_Q2 < 0.05),
  Q3_reject = mean(res_uniform$p_Q3 < 0.05))
```

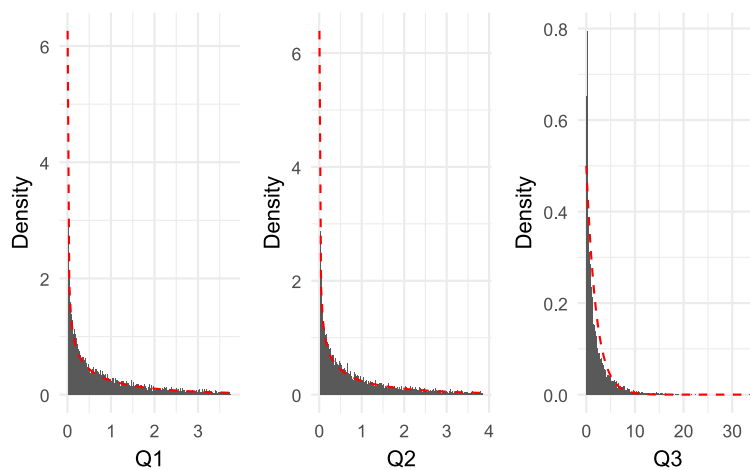
```
Q1_reject Q2_reject Q3_reject
0.0476    0.0505    0.0796
```

```
res_uniform |>
  ggplot() +
    geom_histogram(aes(x = Q1, y = after_stat(density)), bins = 200) +
    geom_function(fun = dchisq, args = list(df = 1), color = "red", lty = 2) +
    xlim(c(quantile(res_uniform$Q1, 0.05), quantile(res_uniform$Q1, 0.95))) +
    labs(x = "Q1", y = "Density") +
    theme_minimal() +
res_uniform |>
  ggplot() +
    geom_histogram(aes(x = Q2, y = after_stat(density)), bins = 200) +
```

```

geom_function(fun = dchisq, args = list(df = 1), color = "red", lty = 2) +
xlim(c(quantile(res_uniform$Q2, 0.05), quantile(res_uniform$Q2, 0.95))) +
labs(x = "Q2", y = "Density") +
theme_minimal() +
res_uniform |>
ggplot() +
geom_histogram(aes(x = Q3, y = after_stat(density)), bins = 200) +
geom_function(fun = dchisq, args = list(df = 2), color = "red", lty = 2) +
labs(x = "Q3", y = "Density") +
theme_minimal()

```



We can run a simulation now where we generate bivariate skewed t values, apply the Rosenblatt transformation with the true parameters, and then compute our test statistic using the interpoint distances. The rejection rate is nominal, with about 5% of samples being rejected.

```

sim_interdistance <- function(i) {
  new_draws <- rtskewt(n = 1e3, mu = mu, sigmas = list(variance),
                      skew = skew, nu = nu)

  U <- tensor_rosenblatt(draws = new_draws, model = "skewt",
                        parameters = list(mu = mu, sigmas = list(variance), skew
= skew, nu = nu))

  interpoint_dist(unclass(U))
}

library(furrr)

```

```
plan(multisession, workers = 8)

res_skewt_params <- future_map_dfr(1:500, sim_interdistance, .progress = TRUE,
                                   .options = furrr_options(seed = TRUE))

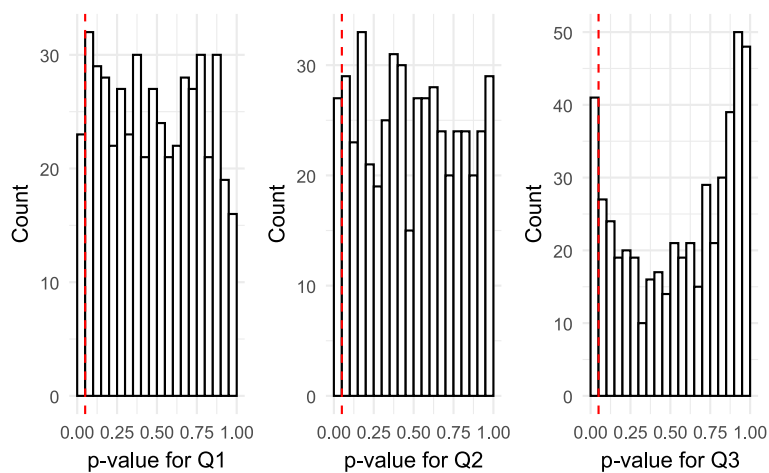
saveRDS(res_skewt_params, "res_skewt_params.rds")
```

```
res_skewt_params <- readRDS("res_skewt_params.rds")

c(Q1_reject = mean(res_skewt_params$p_Q1 < 0.05),
  Q2_reject = mean(res_skewt_params$p_Q2 < 0.05),
  Q3_reject = mean(res_skewt_params$p_Q3 < 0.05))
```

```
Q1_reject Q2_reject Q3_reject
0.046     0.054     0.082
```

```
ggplot() +
  geom_histogram(aes(x = res_skewt_params$p_Q1), binwidth = 0.05, boundary = 0,
color = "black", fill = "white") +
  geom_vline(xintercept = 0.05, color = "red", lty = 2) +
  labs(x = "p-value for Q1", y = "Count") +
ggplot() +
  geom_histogram(aes(x = res_skewt_params$p_Q2), binwidth = 0.05, boundary = 0,
color = "black", fill = "white") +
  geom_vline(xintercept = 0.05, color = "red", lty = 2) +
  labs(x = "p-value for Q2", y = "Count") +
ggplot() +
  geom_histogram(aes(x = res_skewt_params$p_Q3), binwidth = 0.05, boundary = 0,
color = "black", fill = "white") +
  geom_vline(xintercept = 0.05, color = "red", lty = 2) +
  labs(x = "p-value for Q3", y = "Count")
```



We can repeat this diagnostic with the purposefully incorrect variance. Now, we expect the test to reject frequently as the specified distribution model is not accurate. This means the Rosenblatt transformation is not able to fully transform the data back into a 2-dimensional uniform on the unit hypercube. As expected, the test rejects every single time as the data is not uniform.

```
sim_interdistance_wrong <- function(i) {
  new_draws <- rtskewt(n = 1e3, mu = mu, sigmas = list(variance),
                      skew = skew, nu = nu)

  U_wrong <- tensor_rosenblatt(draws = new_draws, model = "skewt",
                              parameters = list(mu = mu, sigmas
= list(wrong_variance),
                              skew = skew, nu = nu))

  interpoint_dist(unclass(U_wrong))
}

res_skewt_wrong <- future_map_dfr(1:500, sim_interdistance_wrong,
                                .progress = TRUE, .options = furrr_options(seed
= TRUE))

saveRDS(res_skewt_wrong, "res_skewt_wrong.rds")
```

```
res_skewt_wrong <- readRDS("res_skewt_wrong.rds")

c(Q1_reject = mean(res_skewt_wrong$p_Q1 < 0.05),
```

```
Q2_reject = mean(res_skewt_wrong$p_Q2 < 0.05),
Q3_reject = mean(res_skewt_wrong$p_Q3 < 0.05))
```

```
Q1_reject Q2_reject Q3_reject
      1      1      1
```

Uniformity test for each marginal

The interpoint distance assesses whether the joint distribution of the Rosenblatt transformed draws are uniform on the d -dimensional unit hypercube. However, we also want to know if they are marginally distributed as $\text{Uniform}(0, 1)$ and mutually independent.

```
test_marginal <- function(draws) {
  d_dims <- length(dim(draws))

  draws_array <- unclass(draws)

  p_marginal <- apply(draws_array, MARGIN = 2:d_dims,
                      FUN = function(z) ks.test(z, "punif", min = 0, max
= 1)$p.value)

  # adjust for multiple comparisons
  p.adjust(p_marginal, method = "BH")
}

test_marginal(res_whitened)
```

```
      u1      u2
0.1879406 0.1879406
```

```
test_marginal(res_whitened_wrong)
```

```
      u1      u2
1.258834e-149 4.099284e-06
```

Account for MLEs

When running the interpoint distance test and the marginal tests, we used the true parameters used to simulate the data. However, in practice, we would not know these. Instead, we would use the MLEs which are computed from the EM algorithm. This requires an adjustment to the test statistic.

The interpoint distance is an order-two U-statistic. U-statistics are statistics that average a function applied to all tuples of a fixed size. It is an order-two statistic since it is averaging the Euclidean distance, which is a square function.

So the function is defined as

$$h_1(X_i, X_j) = ||X_i - X_j||^2$$

And then the interpoint distance is

$$\hat{\delta} = \binom{n}{2}^{-1} \sum_{i < j} h_1(X_i, X_j)$$

Step by step process

Let us say that $X_1, \dots, X_n \sim \text{Tensor Variate Skewed } T(\mathcal{M}, \mathcal{A}, \bigotimes_{i=1}^d \Sigma_i, \nu)$.

1. We use `tensor_mle` to obtain the MLEs, denoted as $\hat{\theta}$.
2. Compute the information matrix for $\hat{\theta}$.
3. Apply the Rosenblatt transformation to remove the independence structure, transforming the draws to uniform.
4. Compute the regular interpoint distances with no adjustment yet.
5. Calculate the first-order projections.
6. Estimate the corrected covariances.
7. From the adjusted test statistic.

```
# Accumulate the score cross-products needed for the MLE correction.
# Only one score chunk is held in memory at a time.
tensor_score_crossproducts <- function(draws, Z, fit,
                                         model = "skewt",
                                         chunk_size = 256L) {
  model <- match.arg(model, c("normal", "skewt", "vargamma",
                              "invgauss", "genhyper"))

  n <- n_draws(draws)
  q <- prod(draw_shape(draws))

  if (!is.matrix(Z) || nrow(Z) != n || ncol(Z) != q) {
    stop("Z must be an n x prod(draw_shape(draws)) matrix.")
  }
  if (length(chunk_size) != 1L || !is.finite(chunk_size) ||
      chunk_size < 1L || chunk_size != as.integer(chunk_size)) {
    stop("chunk_size must be one positive integer.")
  }
  chunk_size <- as.integer(chunk_size)

  # Retain only sums needed for H, G, and I.
```

```

sum_a <- 0
sum_a2 <- 0
sum_s <- NULL
sum_as <- NULL
sum_ss <- NULL

for (start in seq.int(1L, n, by = chunk_size)) {
  end <- min(start + chunk_size - 1L, n)
  index <- start:end

  # Scores are calculated only for this chunk.
  score_chunk <- tensor_scores(
    draws = slice_draws(draws, index),
    model = model,
    fit = fit
  )
  score_chunk <- as.matrix(score_chunk)

  # First-order projection for Q1.
  z_chunk <- Z[index, , drop = FALSE]
  a_chunk <- rowSums(z_chunk^2 - z_chunk + 1 / 6)

  if (is.null(sum_s)) {
    p <- ncol(score_chunk)
    sum_s <- numeric(p)
    sum_as <- numeric(p)
    sum_ss <- matrix(0, nrow = p, ncol = p)
  }

  sum_a <- sum_a + sum(a_chunk)
  sum_a2 <- sum_a2 + sum(a_chunk^2)
  sum_s <- sum_s + colSums(score_chunk)
  sum_as <- sum_as + drop(crossprod(a_chunk, score_chunk))
  sum_ss <- sum_ss + crossprod(score_chunk)
}

# Center by moment identities without making a second pass.
mean_a <- sum_a / n
mean_s <- sum_s / n

H_hat <- sum_a2 / n - mean_a^2
G_hat <- sum_as / n - mean_a * mean_s
I_hat <- sum_ss / n - tcrossprod(mean_s)

```

```

I_hat <- (I_hat + t(I_hat)) / 2

list(H = H_hat, G = G_hat, I = I_hat)
}

adjusted_q1 <- function(draws, theta_hat,
                        rosenblatt_fun,
                        model = "skewt",
                        chunk_size = 256L,
                        U = NULL,
                        n_draws_fun = n_draws) {
  n <- n_draws_fun(draws)

  # Reuse precomputed Rosenblatt values when available.
  if (is.null(U)) {
    U <- rosenblatt_fun(draws, theta_hat)
  }

  Z <- matrix(unclass(U), nrow = n)
  q <- ncol(Z)

  # Raw Q1 U-statistic: mean squared pairwise distance minus q/6.
  row_sq <- rowSums(Z^2)
  pair_sum <- n * sum(row_sq) - sum(colSums(Z)^2)
  U1 <- pair_sum / choose(n, 2) - q / 6

  moments <- tensor_score_crossproducts(
    draws = draws,
    Z = Z,
    fit = theta_hat,
    model = model,
    chunk_size = chunk_size
  )

  # MLE-adjusted covariance; nu = 2 for pairwise interpoint distances.
  beta <- tryCatch(
    solve(moments$I, moments$G),
    error = function(e) {
      stop("The information matrix is singular or ill-conditioned.")
    }
  )

  Sigma_hat <- 4 * (moments$H - sum(moments$G * beta))

```

```

if (!is.finite(Sigma_hat) || Sigma_hat <= 0) {
  stop("Estimated adjusted variance is not positive.")
}

statistic <- n * U1^2 / Sigma_hat

list(
  U1 = U1,
  variance_adjusted = Sigma_hat,
  statistic = statistic,
  p.value = pchisq(statistic, df = 1, lower.tail = FALSE)
)
}

rosenblatt_fun <- function(draws, fit) {
  tensor_rosenblatt(
    draws = draws,
    model = "skewt",
    parameters = fit
  )
}

score_fun <- function(draws, fit) {
  tensor_scores(
    draws = draws,
    model = "skewt",
    fit = fit
  )
}

```

```

compute_p_mle <- function(i) {
  bivar_skewt <- rtskewt(n = 1e3, mu = mu, sigmas = list(variance),
    skew = skew, nu = nu)

  theta_hat <- tensor_mle(bivar_skewt, model = "skewt",
    tol = 1e-6)

  # Rosenblatt transform to uniform
  U_hat <- rosenblatt_fun(bivar_skewt, theta_hat)

  # unadjusted for MLEs
  unadj <- interpoint_dist(unclass(U_hat))

```

```

# adjusted for MLEs
result <- adjusted_q1(
  draws = bivar_skewt,
  theta_hat = theta_hat,
  #rosenblatt_fun = rosenblatt_fun,
  model = "skewt",
  chunk_size = 256L,
  U = U_hat
)

tibble(i = i, q1_unadj = unadj$Q1, p_unadj = unadj$p_Q1,
       q1_adj = result$statistic, p_adj = result$p.value)
}

```

```

library(furrr)

plan(multisession, workers = 10)

res_skewt_MLEs <- future_map_dfr(1:500, compute_p_mle,
                                .progress = TRUE, .options = furrr_options(seed
= TRUE))

```

```

res_skewt_MLEs <- readRDS("res_skewt_MLEs.rds")

c(rej_unadj = mean(res_skewt_MLEs$p_unadj < 0.05),
  rej_adj = mean(res_skewt_MLEs$p_adj < 0.05))

```

```

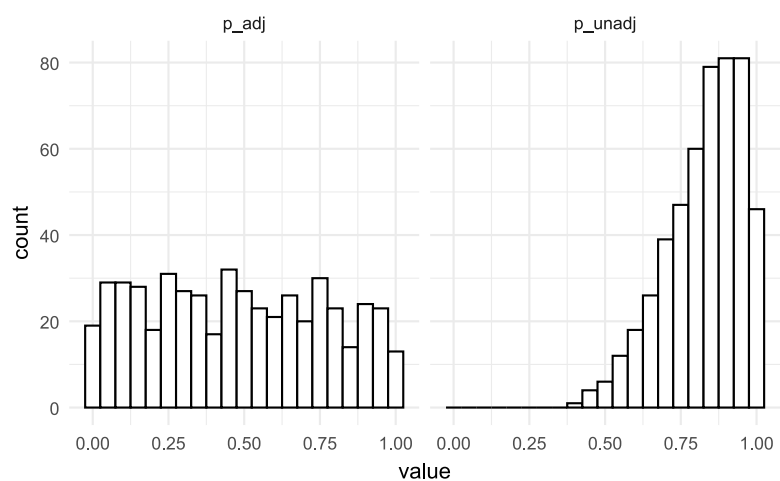
rej_unadj  rej_adj
0.000      0.064

```

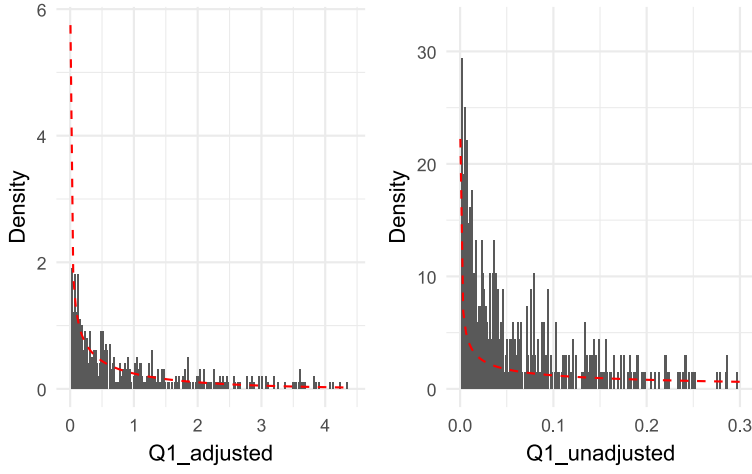
```

res_skewt_MLEs |>
  pivot_longer(cols = c("p_unadj", "p_adj")) |>
  ggplot() +
    geom_histogram(aes(x = value), fill = "white", color = "black", binwidth =
0.05) +
    facet_wrap(~name) +
    theme_minimal()

```



```
res_skewt_MLEs |>
  ggplot() +
    geom_histogram(aes(x = q1_adj, y = after_stat(density)), bins = 200) +
    geom_function(fun = dchisq, args = list(df = 1), color = "red", lty = 2) +
    xlim(c(quantile(res_skewt_MLEs$q1_adj, 0.05),
quantile(res_skewt_MLEs$q1_adj, 0.95))) +
    labs(x = "Q1_adjusted", y = "Density") +
    theme_minimal() +
res_skewt_MLEs |>
  ggplot() +
    geom_histogram(aes(x = q1_unadj, y = after_stat(density)), bins = 200) +
    geom_function(fun = dchisq, args = list(df = 1), color = "red", lty = 2) +
    xlim(c(quantile(res_skewt_MLEs$q1_unadj, 0.05),
quantile(res_skewt_MLEs$q1_unadj, 0.95))) +
    labs(x = "Q1_unadjusted", y = "Density") +
    theme_minimal()
```



Appendix

Interpoint Distances

Let $\mathbf{X}_1, \dots, \mathbf{X}_n \stackrel{\text{iid}}{\sim} U([0, 1]^d)$

We define the interpoint distances as

$$D_{ij} = \|\mathbf{X}_i - \mathbf{X}_j\|^2 = \sum_{k=1}^d (X_{ik} - X_{jk})^2$$

The sample mean of all the interpoint distances is

$$\hat{\delta} = \frac{1}{\binom{n}{2}} \sum_{i < j}^n \|\mathbf{X}_i - \mathbf{X}_j\|^2$$

To find $\text{Var}(D_{ij})$, we need $E[D_{ij}]$ and $E[D_{ij}^2]$.

$$E[D_{ij}] = E \left[\sum_{k=1}^d (X_{ik} - X_{jk})^2 \right] = \sum_{k=1}^d E[X_{ik}^2 - 2X_{ik}X_{jk} + X_{jk}^2]$$

For a Uniform,

$$E[X] = \int_0^1 x f(x) dx = \left[\frac{x^2}{2} \right]_0^1 = \frac{1}{2}$$

$$E[X^2] = \int_0^1 x^2 f(x) dx = \left[\frac{x^3}{3} \right]_0^1 = \frac{1}{3}$$

We are assuming that $\mathbf{X}_i, \dots, \mathbf{X}_n$ are independent so

$$\begin{aligned}
& \sum_{k=1}^d E[X_{ik}^2 - 2X_{ik}X_{jk} + X_{jk}^2] \\
&= \sum_{k=1}^d \left[\frac{1}{3} - 2\left(\frac{1}{2}\right)\left(\frac{1}{2}\right) + \frac{1}{3} \right] \\
&= \sum_{k=1}^d \left(\frac{2}{3} - \frac{1}{2} \right) \\
&= \sum_{k=1}^d \frac{1}{6} = \frac{d}{6}
\end{aligned}$$

So we can obtain $E[\hat{\delta}]$.

\$\$\$

Now we are interested in the variance of this sample mean.

$$\begin{aligned}
\text{Var}(\hat{\delta}) &= \text{Var} \left(\frac{1}{\binom{n}{2}} \sum_{i < j}^n | \mathbf{X}_i - \mathbf{X}_j |^2 \right) \\
&= \left(\frac{1}{\binom{n}{2}} \right)^2 \left[\sum_{i < j}^n \text{Var} (| X_i - X_j |^2) + 2 \sum_{(i < j) < (k < l)} \text{Cov} (| X_i - X_j |^2, | X_k - X_l |^2) \right]
\end{aligned}$$

Now we first consider the variance.

$$\begin{aligned}
\text{Var}(\|X_i - X_j\|^2) &= E[(\|X_i - X_j\|^2)^2] - [E(\|X_i - X_j\|^2)]^2 \\
&= E\left\{\left[\sum_{k=1}^d (X_{ik} - X_{jk})^2\right]^2\right\} - \frac{d}{6^2} \\
&= \sum_{k=1}^d E[(X_{ik} - X_{jk})^4] + 2 \sum_{1 \leq k < l \leq d} E[(X_{ik} - X_{jk})^2 (X_{il} - X_{jl})^2] - \frac{d}{6^2} \\
&= \sum_{k=1}^d E[X_{ik}^4 - 4X_{ik}^3 X_{jk} + 6X_{ik}^2 X_{jk}^2 - 4X_{ik} X_{jk}^3 + X_{jk}^4] + 2 \sum_{1 \leq k < l \leq d} E[(X_{ik} - X_{jk})^2 (X_{il} - X_{jl})^2] - \frac{d}{6^2} \\
&= \sum_{k=1}^d \left[\frac{1}{5} - 4\frac{1}{4}\frac{1}{2} + 6\frac{1}{3}\frac{1}{3} - 4\frac{1}{2}\frac{1}{4} + \frac{1}{5}\right] + 2 \sum_{1 \leq k < l \leq d} E[(X_{ik} - X_{jk})^2 (X_{il} - X_{jl})^2] - \frac{d}{6^2} \\
&= \frac{d}{15} + 2 \sum_{1 \leq k < l \leq d} E[(X_{ik} - X_{jk})^2] E[(X_{il} - X_{jl})^2] - \frac{d}{6^2} \\
&= \frac{d}{15} + 2 \binom{d}{2} \frac{1}{6} \frac{1}{6} - \frac{d}{6^2} \\
&= \frac{d}{15} + \frac{d(d-1)}{36} - \frac{d^2}{36} \\
&= \frac{12d + 5d(d-1) - 5d^2}{180} \\
&= \frac{12d + 5d^2 - 5d - 5d^2}{180} \\
&= \frac{7d}{180}
\end{aligned}$$

Now we consider the covariance.

$$\begin{aligned}
& \sum_{(i < j) < (k < l)} \text{Cov}(|X_i - X_j|^2, |X_k - X_l|^2) \\
&= \sum_{(i < j) < (k < l)} E \left[\left(|X_i - X_j|^2 - \frac{d}{6} \right) \left(|X_k - X_l|^2 - \frac{d}{6} \right) \right] \\
&= \sum_{(i < j) < (k < l)} E \left[|X_i - X_j|^2 |X_k - X_l|^2 - \frac{d}{6} |X_i - X_j|^2 - \frac{d}{6} |X_k - X_l|^2 + \frac{d^2}{36} \right] \\
&= \sum_{(i < j) < (k < l)} \left\{ E[|X_i - X_j|^2 |X_k - X_l|^2] - \frac{d}{6} \frac{d}{6} - \frac{d}{6} \frac{d}{6} + \frac{d^2}{36} \right\} \\
&= \sum_{(i < j) < (k < l)} \left\{ E[|X_i - X_j|^2 |X_k - X_l|^2] - \frac{d^2}{36} \right\} \\
&= \sum_{(i < j) < (k < l)} \left\{ E[|X_i - X_j|^2 |X_k - X_l|^2] - \frac{d^2}{36} \right\}
\end{aligned}$$

There are 3 possible cases here. If $\{i, j\}$ does not overlap with $\{k, l\}$ then the covariance is 0. If $\{i, j\}$ is the same as $\{k, l\}$ then the points are the same and this is the variance. The final possibility is if one of the points overlap. Here, let us assume X_i and X_k are the same.

$$\begin{aligned}
&= \sum_{(i < j) < (k < l)} \left\{ E[|X_i - X_j|^2 |X_k - X_l|^2] - \frac{d^2}{36} \right\} \\
&= \sum_{(i < j) < (k < l)} \left\{ E[|X_i - X_j|^2 |X_k - X_l|^2] - \frac{d^2}{36} \right\}
\end{aligned}$$

We can also compute the sample squared deviation from the hypothesized mean.

$$\begin{aligned}
\text{Var}(D_{ij}) &= E[(D_{ij} - E(D_{ij}))^2] \\
&= E \left[\left(|X_i - X_j|^2 - \frac{d}{6} \right)^2 \right]
\end{aligned}$$

And we estimate this using an empirical estimator

$$\hat{S} = \frac{1}{\binom{n}{2}} \sum_{i < j}^n \left(|X_i - X_j|^2 - \frac{d}{6} \right)^2$$

```

mu <- array(1:24, dim = c(2, 3, 4))
s1 <- crossprod(matrix(rnorm(4), nrow = 2))
s2 <- crossprod(matrix(rnorm(9), nrow = 3))

```

```
s3 <- crossprod(matrix(rnorm(16), nrow = 4))
sigmas <- list(s1, s2, s3)
skew <- array((seq(from = 0, to = 1, length.out = 24)), dim = c(2, 3, 4))
```

```
test_tensor <- function(i) {

  tensor_skewt <- rtskewt(n = 1e3, mu = mu, sigmas = sigmas, skew = skew)

  whitened_tensor <- tensor_rosenblatt(draws = tensor_skewt, model = "skewt",
                                       parameters = list(mu = mu, sigmas = sigmas,
skew = skew, nu = 4))

  interpoint_dist(whitened_tensor)
}

library(furrr)

plan(multisession, workers = 6)
res <- future_map_dfr(1:100, test_tensor, .progress = TRUE,
                     .options = furrr_options(seed = TRUE))

ggplot() +
  geom_histogram(aes(x = res$p_Q1))
```

```
# result <- adjusted_q1(
#   draws = tensor_skewt,
#   theta_hat = list(mu = mu, sigmas = sigmas, skew = skew, nu = 4),
#   rosenblatt_fun = rosenblatt_fun,
#   U = whitened_tensor
# )

result$p.value

ggplot() +
  geom_histogram(aes(x = whitened_tensor[, 1, 1, 1]))

whitened_tensor[, 1, 1, 1]

whitened_tensor[1, 1, 1, 1]
```

```
ggplot() +  
  geom_point(aes)
```

```
library(plotly)  
  
U <- unclass(whitened_tensor)  
  
plot_data <- data.frame(  
  u1 = U[, 1, 1, 1],  
  u2 = U[, 1, 1, 2],  
  u3 = U[, 1, 1, 3]  
)  
  
plot_ly(  
  plot_data,  
  x = ~u1,  
  y = ~u2,  
  z = ~u3,  
  type = "scatter3d",  
  mode = "markers",  
  marker = list(size = 2, opacity = 0.25)  
) |>  
  layout(  
    scene = list(  
      xaxis = list(title = "U[1,1,1]", range = c(0, 1)),  
      yaxis = list(title = "U[1,1,2]", range = c(0, 1)),  
      zaxis = list(title = "U[1,1,3]", range = c(0, 1)),  
      aspectmode = "cube"  
    )  
  )  
)
```